Guia de Mapeamento de Sinais CAN

1. Introdução

Este guia ensina como mapear sinais do arquivo DBC para o display LVGL e como configurar botões para enviar mensagens CAN no HMI Manager.

Arquivos Importantes

Arquivo	Você Edita?	Finalidade
`app_hmi_manager.c`	**SIM**	Implementa lógica RX (CAN → Display) e TX (Botões → CAN)
`can/car_can.h`	**NÃO (só lê)**	Consulta IDs, sinais, structs (gerado por cantools)
`can/car_can.c`	**NÃO**	Implementação gerada (não mexa)
`uiux/widgets.c`	Raramente	Widgets LVGL prontos

2. Fluxo de Dados

Recepção CAN → Display (RX)

```
Barramento CAN
    ↓
FDCAN Driver (kernel)
    ↓
SocketCAN (/dev/can0)
    ↓
can_handler_read()  ← Lê frame bruto
    ↓
process_can_frame()  ← VOCÊ EDITA AQUI
    ↓
    1. Identifica ID da mensagem
    2. Desempacota: car_can_*_unpack()
    3. Decodifica sinal: car_can_*_decode()
    4. Atualiza variável global
    ↓
widgets_set_speed()  ← Atualiza LVGL
widgets_set_voltage()
    ↓
Display LCD
```

Transmissão Botões → CAN (TX)

```
Botão pressionado (IRQ)
    ↓
button_irq_get_pressed()
    ↓
process_button_press()  ← VOCÊ EDITA AQUI
    ↓
    1. Consulta tabela g_button_map[]
    2. Empacota: car_can_*_pack()
    3. Envia: can_handler_send()
    ↓
Barramento CAN
```

3. Entendendo o car_can.h

> **IMPORTANTE:** O arquivo `car_can.h` é gerado automaticamente pelo cantools a partir do DBC. Você NÃO edita este arquivo, apenas o consulta.

Estrutura do car_can.h

Parte 1: IDs das Mensagens

```
/* Frame ids */
#define CAR_CAN_INVERTER_DEMANDERS_FRAME_ID (0xc09fefeu)
#define CAR_CAN_SYSTEM_VOLTAGES_FRAME_ID (0xc06fe00u)
#define CAR_CAN_DRIVER_COMMANDS_FRAME_ID (0xc03fe32u)
```

Como usar: Copia o nome exato para usar no `if (id == ...)`

Parte 2: Tipo de ID (Standard vs Extended)

```
#define CAR_CAN_SYSTEM_VOLTAGES_IS_EXTENDED (1)  /* 1 = Extended (29 bits) */
```

Parte 3: Estruturas das Mensagens

```
struct car_can_system_voltages_t {
    uint16_t meas12_v;           /* 12V rail voltage */
    uint8_t meas5_v;             /* 5V rail voltage */
    uint16_t meas_dc_link;       /* DC link voltage */
    uint16_t meas_batt_voltage;  /* Battery voltage */
};
```

Informações importantes:

- **Nome da struct:** `car_can_system_voltages_t`
- **Sinais disponíveis:** `meas12_v`, `meas5_v`, `meas_dc_link`, `meas_batt_voltage`
- **Tipo de cada campo:** `uint16_t`, `uint8_t`, etc

Parte 4: Funções Encode/Decode

```
/* Decode: valor bruto → valor físico */
double car_can_system_voltages_meas_batt_voltage_decode(uint16_t value);

/* Encode: valor físico → valor bruto */
uint16_t car_can_system_voltages_meas_batt_voltage_encode(double value);
```

Quando usar:

- **RX:** Use `decode()` para converter valor do CAN em valor físico
- **TX:** Use `encode()` para converter valor físico em valor do CAN

4. Recepção: CAN → Display

Passo a Passo Completo

> **PASSO 1:** Descobrir qual mensagem tem o sinal desejado

Abra `car_can.h` e procure por palavras-chave relacionadas ao sinal.

> **Exemplo:** Quero mostrar a velocidade do motor no display.
>
> Procuro por: `speed`, `rpm`, `velocity`
>
> Encontrei (hipotético):

```
struct car_can_motor_status_t {
    uint16_t motor_rpm;      /* Rotação do motor */
    uint8_t motor_temp;      /* Temperatura */
    uint8_t motor_state;     /* Estado */
};

#define CAR_CAN_MOTOR_STATUS_FRAME_ID (0x18FF1234u)
```

PASSO 2: Verificar a escala do sinal

Procure nos comentários do `car_can.h`:

```
/**
 * Range: 0..65535 (0..6553.5 RPM)
 * Scale: 0.1
 * Offset: 0
 */
uint16_t motor_rpm;
```

Interpretação:

- Valor bruto no CAN: 0 a 65535
- Valor físico: 0.0 a 6553.5 RPM
- Conversão: `RPM_físico = valor_bruto × 0.1`

PASSO 3: Editar app_hmi_manager.c

Localize a função `process_can_frame()` (linha ~100-200) e adicione:

```
/* Motor RPM */
if (id == CAR_CAN_MOTOR_STATUS_FRAME_ID)
{
    struct car_can_motor_status_t msg;

    ret = car_can_motor_status_unpack(&msg, frame->data,
                                      frame->can_dlc);
    if (ret == 0)
    {
        /* Decodifica RPM (bruto → físico) */
        double rpm = car_can_motor_status_motor_rpm_decode(msg.motor_rpm);

        /* Atualiza variável global */
        g_speed = (int)rpm;

        /* Atualiza display */
        widgets_set_speed(g_speed);
    }
}
```

PASSO 4: Compilar e testar

```
make -j$(nproc)
make flash
```

Teste no Linux:

```
# Envia motor_rpm = 3000 RPM
# 3000 / 0.1 = 30000 = 0x7530
# Little-endian: 30 75

cansend can0 18FF1234#3075000000000000
```

Template Genérico para RX

```
if (id == CAR_CAN_[NOME_MENSAGEM]_FRAME_ID)
{
    struct car_can_[nome_mensagem]_t msg;
```

```
    ret = car_can_[nome_mensagem]_unpack(&msg, frame->data,
                                         frame->can_dlc);
    if (ret == 0)
    {
        /* Decodifica sinal */
        double valor = car_can_[nome_mensagem]_[nome_sinal]_decode(
                        msg.[nome_sinal]);

        /* Atualiza variável */
        g_[variavel] = (int)valor;

        /* Atualiza widget */
        widgets_set_[widget](g_[variavel]);
    }
}
```

5. Transmissão: Botões → CAN

Arquitetura de Botões

O sistema usa uma **tabela de configuração** para mapear botões → mensagens CAN.

Estrutura de Configuração

```
struct button_config_s
{
  uint32_t msg_id;       /* CAN message ID */
  bool extended;         /* Extended ID flag */
  uint8_t data[8];       /* Pre-packed CAN data */
  uint8_t dlc;           /* Data Length Code (0-8) */
};
```

Criar Tabela de Mapeamento

No topo de `app_hmi_manager.c`, após os includes:

```
/* Button mapping table */
static const struct button_config_s g_button_map[] =
{
  /* BTN0: Hazard lights */
  {
    .msg_id = CAR_CAN_BUTTON_COMMANDS_FRAME_ID,
    .extended = CAR_CAN_BUTTON_COMMANDS_IS_EXTENDED,
    .data = {0x00, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00},
    .dlc = 2
  },

  /* BTN1: Left turn signal */
  {
    .msg_id = CAR_CAN_BUTTON_COMMANDS_FRAME_ID,
    .extended = CAR_CAN_BUTTON_COMMANDS_IS_EXTENDED,
    .data = {0x01, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00},
    .dlc = 2
  },

  /* BTN2: Right turn signal */
  {
    .msg_id = CAR_CAN_BUTTON_COMMANDS_FRAME_ID,
    .extended = CAR_CAN_BUTTON_COMMANDS_IS_EXTENDED,
    .data = {0x02, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00},
    .dlc = 2
  },

  /* Adicione mais botões conforme necessário */
};

#define NUM_BUTTONS (sizeof(g_button_map) / sizeof(g_button_map[0]))
```

Calcular Valores dos Bytes

Método 1: Manual (rápido)

Se a mensagem é simples, calcule direto:

> **Exemplo:** Mensagem `ButtonCommands` tem 2 bytes:

- Byte 0: `button_id` (0-255)
- Byte 1: `button_state` (0=released, 1=pressed)

Para BTN3 (horn):

```
.data = {0x03, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}
         ^^^^  ^^^^
         |     └─ button_state = 1 (pressed)
         └─────── button_id = 3 (horn)
```

Método 2: Usar Funções DBC (recomendado)

```
/* Helper para BTN3 */
static void pack_btn3(uint8_t *data, size_t size)
{
  struct car_can_button_commands_t msg;

  car_can_button_commands_init(&msg);
  msg.button_id = 3;      /* Horn */
  msg.button_state = 1;   /* Pressed */

  car_can_button_commands_pack(data, &msg, size);
}
```

6. Exemplo Prático Completo

Cenário

Implementar:

1. Display mostra tensão da bateria (já existe no DBC atual)
2. Botão 5 envia comando de reset

Parte 1: Tensão da Bateria (RX)

Consultar car_can.h

```
struct car_can_system_voltages_t {
    uint16_t meas12_v;
    uint8_t meas5_v;
    uint16_t meas_dc_link;
    uint16_t meas_batt_voltage;  /* ← ESTE */
};

#define CAR_CAN_SYSTEM_VOLTAGES_FRAME_ID (0xc06fe00u)
```

Ver Escala

```
/**
 * Range: 0..65535 (0..655.35 V)
 * Scale: 0.01
 * Offset: 0
 */
uint16_t meas_batt_voltage;
```

Exemplo: CAN = 4850 → Físico = 4850 × 0.01 = 48.5V

Implementar

```
/* Battery voltage */
if (id == CAR_CAN_SYSTEM_VOLTAGES_FRAME_ID)
{
    struct car_can_system_voltages_t msg;

    ret = car_can_system_voltages_unpack(&msg, frame->data,
                                         frame->can_dlc);
```

```
    if (ret == 0)
    {
        /* Decodifica (retorna Volts) */
        double voltage = car_can_system_voltages_meas_batt_voltage_decode(
                         msg.meas_batt_voltage);

        /* Widget espera decivolts (0.1V), multiplica por 10 */
        g_voltage = (int)(voltage * 10.0);

        widgets_set_voltage(g_voltage);
    }
}
```

Testar

```
# Envia 48.5V
# 48.5 / 0.01 = 4850 = 0x12F2
# Little-endian: F2 12

cansend can0 0C06FE00#000000000000F212
```

Parte 2: Botão Reset (TX)

Adicionar na Tabela

```
/* BTN5: Reset command */
{
  .msg_id = CAR_CAN_DEBUG_DEMANDS_FRAME_ID,
  .extended = CAR_CAN_DEBUG_DEMANDS_IS_EXTENDED,
  .data = {0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00},
  .dlc = 1
},
```

7. Troubleshooting

Display não atualiza

Diagnóstico:

1. Ative trace CAN: `hmi_ctl can --trace show`
2. Se mensagem aparece no trace mas display não muda → Erro em `process_can_frame()`
3. Se não aparece no trace → Problema no CAN físico

Solução:

- Verifique se ID está correto
- Adicione `printf()` temporário para debug
- Verifique se função `decode()` está sendo chamada

Valor errado no display

Causa comum: Esqueceu de decodificar ou aplicou escala errada

```
/* ERRADO */
g_voltage = (int)msg.voltage_v;  /* Usa valor bruto */

/* CERTO */
double v = car_can_system_voltages_meas_batt_voltage_decode(msg.voltage_v);
g_voltage = (int)(v * 10.0);
```

Botão não envia CAN

Diagnóstico:

1. Adicione `printf("Button %d\n", button);` em `process_button_press()`
2. Verifique se `NUM_BUTTONS` está correto

3. Verifique se tabela tem entrada para esse botão

8. Checklist

Para RX (CAN → Display)

- ☐ Consultei `car_can.h` e encontrei mensagem/sinal corretos
- ☐ Copiei ID da mensagem exato
- ☐ Adicionei `if (id == ...)` em `process_can_frame()`
- ☐ Chamei `*_unpack()` corretamente
- ☐ Chamei `*_decode()` para obter valor físico
- ☐ Atualizei widget com `widgets_set_*()`
- ☐ Testei com `cansend` no Linux
- ☐ Display atualizou corretamente

Para TX (Botões → CAN)

- ☐ Criei entrada na tabela `g_button_map[]`
- ☐ Defini `msg_id`, `extended`, `data`, `dlc`
- ☐ Calculei bytes corretamente (manual ou com `_pack()`)
- ☐ Compilei sem erros
- ☐ Testei pressionando botão físico
- ☐ Mensagem apareceu no trace CAN
- ☐ Destino (ECU) recebeu corretamente